AUTOMATED WEB SECURITY SCANNER

Cybersecurity Project Report

Submitted by

B Tulasi

College

Sri Vidyaniketan Degree College

Technology

Python | Tkinter | Requests

Academic project documentation | October 2026

Abstract

The Automated Web Security Scanner is a desktop application developed in Python to review selected web security configuration signals. It uses the Requests library to retrieve a web response and Tkinter to provide a graphical interface. The application displays HTTPS status, the presence of selected HTTP security headers, basic Cross-Origin Resource Sharing (CORS) response information, and cookie attributes when cookies are set.

The application calculates a simple configuration score from HTTPS use and the presence of six selected security headers. Findings can be exported as a text report. The project is intended for learning and preliminary authorized configuration review; it does not demonstrate exploitability or replace a full security assessment.

Keywords

Python; cybersecurity; HTTP response headers; HTTPS; CORS; Tkinter; security configuration; reporting.

Project at a glance

Item	Description
Application type	Python desktop GUI
Input	Website URL
Output	On-screen findings and saved .txt report
Dependencies	requests (Tkinter supplied with typical Windows Python installs)
Scope	Passive inspection of an authorized HTTP response

1. Introduction

Web applications are commonly accessed through browsers and exchange data using HTTP or HTTPS. Server response headers and cookies can influence how browsers handle content, transport security, framing, permissions, and cross-origin access. Reviewing these settings is a useful introduction to defensive web security.

Manually inspecting response headers requires familiarity with browser developer tools or command-line utilities. This project collects a few observable signals in a single, beginner-friendly interface. It is designed to demonstrate HTTP requests, response parsing, conditional logic, and report generation.

1.1 Problem statement

Students and beginner developers may find it difficult to interpret common web security settings. A small tool that summarizes selected settings can support learning and help identify areas for manual follow-up.

1.2 Project scope

- Accept a URL and request a page over HTTP(S).
- Show whether the final response URL uses HTTPS.
- Check the presence of six named response headers.
- Summarize basic CORS response header values and cookie flags.
- Display an indicative configuration score and export a text report.

2. Objectives and Requirements

2.1 Objectives

• Create an easy-to-use desktop interface with a URL input field.

• Automate retrieval of response metadata using Python Requests.

• Explain missing configuration signals without claiming a proven vulnerability.

• Provide a readable result summary for student demonstration.

• Allow users to save the report locally.

2.2 Functional requirements

ID	Requirement
FR-01	Accept a website URL from the user.
FR-02	Perform an HTTP(S) GET request with a timeout.
FR-03	Display final URL and HTTP status.
FR-04	Check HTTPS, selected security headers, CORS and cookies.
FR-05	Display a score and allow report export.

2.3 Non-functional requirements

The application should remain understandable to a beginner, use a simple local installation, avoid storing credentials, and report network errors without crashing. Network requests may take several seconds; the current implementation runs the request in the UI thread and can temporarily make the interface unresponsive.

3. Existing and Proposed System

3.1 Existing approach

A manual approach uses browser developer tools to inspect network requests and response headers. It offers flexibility but requires users to know where settings appear and how to interpret them.

3.2 Proposed approach

The proposed scanner accepts a URL, sends a request, reads response metadata, and displays structured findings. It also provides a one-click option to save the same findings as a text file.

3.3 Comparison

Aspect	Manual inspection	Proposed scanner
Interface	Browser developer tools	Tkinter desktop window
Header review	Inspect individually	Six named headers checked
Report	Manually record observations	Save a generated .txt file
Interpretation	User-driven	Simple pass/warning/info labels
Coverage	Can inspect multiple requests	Primarily one GET response

4. System Design and Architecture

4.1 Processing flow

Input URL → Normalize and validate → Requests GET → Read response metadata → Run checks → Display results → Save text report

4.2 Main components

• GUI layer: Tkinter labels, entry field, buttons, results text area and scrollbar.

• Network layer: Requests GET with timeout and redirects enabled.

• Analysis layer: URL scheme, header presence, CORS values and cookie attributes.

• Reporting layer: creates readable text and writes it to a user-selected location.

4.3 Data flow

The user supplies a URL. The application makes a network request and receives an HTTP response. It extracts only information needed for the checks and formats results into an in-memory string. The report is written to disk only when the user selects the save option.

4.4 Trust boundary

Websites are external systems. Response headers are untrusted data; the application should display them as text and avoid treating a site response as executable instructions. The tool is intended only for authorized targets.

5. Technology Stack and Setup

5.1 Software

Technology	Role
Python 3	Application language
Tkinter	Desktop GUI widgets
Requests	HTTP(S) request and response handling
urllib.parse	URL parsing
datetime	Timestamp in report
VS Code	Development environment

5.2 Hardware

A standard Windows laptop or desktop with an internet connection is sufficient. No database server or cloud account is required to run the basic program.

5.3 Installation

Create a project folder containing web_scanner.py, requirements.txt and README.md. The requirements file specifies the Requests dependency. In the VS Code terminal run:

py -m pip install -r requirements.txt

Then start the program with: py web_scanner.py. On some systems, python can be used instead of py.

6. Implementation: Security Checks

6.1 HTTPS

The application checks whether the final URL uses the https scheme. This is a transport-related observation; it does not independently evaluate certificate policy, TLS versions, or the security of every resource loaded by a page.

6.2 HTTP response headers

• Content-Security-Policy: can restrict sources of active content and other resources.

• Strict-Transport-Security: can instruct browsers to prefer HTTPS for future connections.

• X-Content-Type-Options: helps reduce MIME type confusion.

• X-Frame-Options: can restrict framing; CSP frame-ancestors is another relevant control.

• Referrer-Policy: influences referrer information shared by browsers.

• Permissions-Policy: controls access to selected browser features.

A present header is not necessarily correctly configured. The tool checks presence rather than validating the full value or its effectiveness.

6.3 CORS and cookies

The application reads Access-Control-Allow-Origin and Access-Control-Allow-Credentials when present. A wildcard origin is not automatically a vulnerability; suitability depends on resource sensitivity and authentication. Cookie attributes Secure, HttpOnly and SameSite are checked for cookies set in the inspected response.

7. Scoring, Interpretation and Limitations

7.1 Score calculation

The basic score is calculated as: (number of passed checks / seven) x 100. The seven checks consist of HTTPS plus six selected security header presence checks. CORS and cookie observations do not contribute to this score.

7.2 Score categories

Score range	Label used by application
80–100%	Low configuration risk
50–79.99%	Medium configuration risk
Below 50%	High configuration risk

7.3 Important interpretation

These labels are educational heuristics, not validated security risk ratings. A missing header may be irrelevant to a particular application, and a present header may still be misconfigured. The scanner does not test vulnerabilities such as SQL injection, authentication bypass, cross-site scripting, or business logic flaws.

7.4 Known limitations

• The checks are based primarily on one GET response and may not represent all pages.

• CORS behavior is not tested using varied Origin headers or preflight requests.

• The application does not inspect all redirect hops or loaded subresources.

• The request is synchronous, so the interface can pause during network access.

• Scoring treats each selected header equally, regardless of site context.

8. Testing and Observed Results

8.1 Test approach

The student launched the Tkinter interface, entered https://example.com, selected Scan Website, and observed the result panel. The provided screenshot shows multiple missing-header warnings, CORS and cookie information, and a calculated configuration score.

8.2 Observed demonstration

Observation	Displayed result
Input URL	https://example.com
Security header review	Six missing-header warnings shown
CORS	No Access-Control-Allow-Origin header found
Cookies	No cookies set in this response
Configuration score	14.29%
Assessment label	HIGH CONFIGURATION RISK

These observations are from the student-provided screenshot of a single run. Results can vary with server responses, time and network conditions. The screenshot did not show every line of the response, so HTTPS and HTTP status are not independently transcribed here.

8.3 Additional verification

The student also saved a text scan report and successfully installed the Requests dependency using the requirements file. The application GUI and report display were demonstrated in the conversation. No claim is made that the scanner has undergone formal penetration testing.

9. User Guide and Submission Materials

9.1 How to use

• Open the project folder in VS Code.

• Install dependencies with: py -m pip install -r requirements.txt

• Run: py web_scanner.py

• Enter an authorized website URL in the text field.

• Click SCAN WEBSITE and wait for results.

• Review HTTPS, header, CORS and cookie observations.

• Click DOWNLOAD REPORT and save Security_Scan_Report.txt.

9.2 Project file structure

automated_web_security_scanner/
web_scanner.py — Python source code
requirements.txt — Python package requirement
README.md — Setup and usage documentation
Security_Scan_Report.txt — Example saved results
Project_Report.pdf — This report

9.3 Form upload mapping

Submission field	File or value
Code File	web_scanner.py
Requirements File	requirements.txt
Project Report	Project_Report.pdf
README File	README.md
GitHub Repository	URL of the repository created by the student

10. Future Enhancements and Conclusion

10.1 Future enhancements

• Add context-sensitive validation of header values rather than presence alone.

• Support a controlled list of authorized pages with clear request limits.

• Improve CORS review using safe, permission-based Origin and preflight tests.

• Move network activity to a worker thread so the GUI remains responsive.

• Add export to PDF or CSV and include screenshots of actual runs.

• Provide clearer guidance on which findings require manual review.

10.2 Conclusion

The Automated Web Security Scanner demonstrates a practical beginner cybersecurity workflow: retrieving an HTTP response, inspecting selected configuration signals, displaying findings in a desktop interface, and saving a report. The project strengthens understanding of Python GUI development, HTTP headers, HTTPS, CORS, cookies and basic reporting.

Its findings should be treated as starting points for manual analysis, not proof that a website is secure or vulnerable. Used responsibly on authorized websites, the project is a useful educational tool and a foundation for more comprehensive defensive security checks.

References and further reading

• Python documentation: tkinter and urllib.parse modules.

• Requests documentation: HTTP requests, responses and exceptions.

• OWASP Secure Headers Project and OWASP Web Security Testing Guide.

• MDN Web Docs: HTTP headers, CORS and Set-Cookie.

Prepared for: B Tulasi | Sri Vidyaniketan Degree College